Chapter-4

Software Testing & Maintenance

Software Testing

Software Testing is a method to check whether the actual **software product matches expected requirements** and to ensure that software product is **defect free**.

It involves **execution of software/system components** using **manual or automated tools** to evaluate one or more **properties of interest**.

The purpose of software testing is to **identify errors, gaps or missing requirements** in contrast to actual requirements.

Software Testing **is important** because if there are any bugs or errors in the software, **it can be identified early and can be solved before delivery of the software product**.

Strategic Approach to Software Testing

A **generic characteristics of software testing strategies** are:

- For effective testing, **technical reviews must be conducted**. By doing this, many errors will be eliminated before testing commences.
- Testing **begins at the component level and works outward toward the integration of the entire computer-based system**.
- **Different testing techniques are appropriate** for different software engineering approaches and at different points in time.
- Testing is **conducted by the developer** of the software and (for large projects) an **independent test group**.
- **Testing and debugging are different activities**, but debugging must be accommodated in any testing strategy.

Verification and Validation

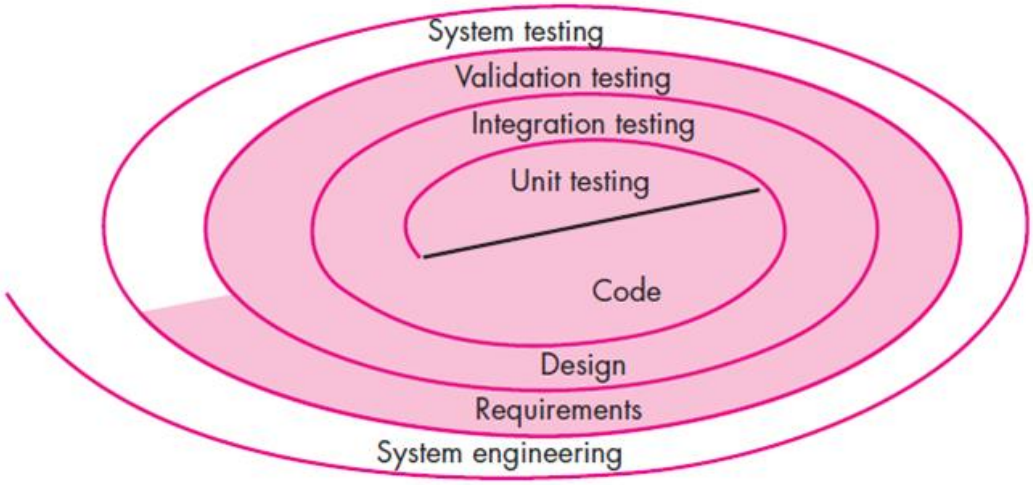
Software testing is one element of a broader topic that is often referred to as verification and validation (V&V). Verification refers to the set of tasks that ensure that software correctly implements a specific function. Validation refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements. Boehm [Boe81] states this another way:

Verification: "Are we building the product, right?"

Validation: "Are we building the right product?"

Software Testing Strategy

The software process may be viewed as the spiral illustrated in Figure. Initially, system engineering defines the role of software and leads to software requirements analysis, where the information domain, function, behavior, performance, constraints, and validation criteria for software are established. Moving inward along the spiral, you come to design and finally to coding. To develop computer software, you spiral inward (counter clockwise) along streamlines that decrease the level of abstraction on each turn.



TEST STRATEGIES FOR CONVENTIONAL SOFTWARE

Testing type	Testing focused on	Description
Unit testing	Code (Module, Object)	In **unit testing**, tests focus on each component as a unit. Specific **paths in a component's control structure are tested** to ensure complete coverage and maximum error detection.
Integration testing	Design	**Integration testing** is performed on **components that integrated** to form the complete software package. Test case design techniques **focuses on inputs and outputs** to ensure coverage of major control paths.
Validation testing	Requirements	**Validation testing** provides final assurance that **software meets all informational, functional, behavioral, and performance requirements.**
System testing	System	Software, once validated, must be combined with other system elements (e.g., hardware, people, databases). **System testing** verifies that **all elements mesh properly and that overall system function/performance** is achieved.

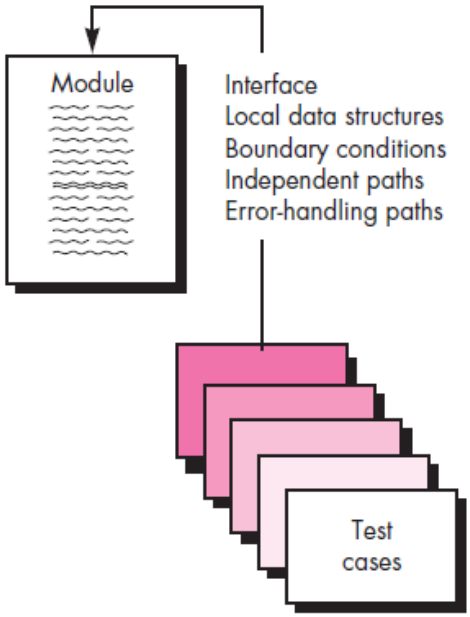
1. Unit testing

In Unit testing -

- testing is performed on smallest unit of software design – **the software component or module.**
- important **control paths are tested** to uncover errors within the boundary of the module**.**
- focus is on the **internal processing logic and data structures** within the boundaries of a component.
- testing can be conducted **in parallel for multiple components**.

Unit-test considerations.

Unit tests are illustrated schematically in Figure. The module interface is tested to ensure that information properly flows into and out of the program unit under test. Local data structures are examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution. All independent paths through the control structure are exercised to ensure that all statements in a module have been executed at least once. Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing. And finally, all error-handling paths are tested.



Unit-test procedures

Unit testing is normally considered as an adjunct to the coding step. The design of unit tests can occur before coding begins or after source code has been generated. A review of design information provides guidance for establishing test cases that are likely to uncover errors in each of the categories discussed earlier. Each test case should be coupled with a set of expected results. Because a component is not a stand-alone program, driver and/or stub software must often be developed for each unit test. The unit test environment is illustrated in
Figure.

In most applications a ***driver*** is nothing more than a "main program" that accepts test case data, passes such data to the component (to be tested), and prints relevant results.

Stubs serve to replace modules that are subordinate (invoked by) the component to be tested. A stub or "dummy subprogram" uses the subordinate module's interface, may do minimal data manipulation, prints verification of entry, and returns control to the module undergoing testing.

Use of stub

It is always preferable, to perform testing, in parallel, to the development process. Thus, **Module-A** will be tested, as soon as, it develops. However, to carry out and validate the testing procedures in respect of module-A, there urges the need of **Module-B**, which is not yet developed. The expected functionality of the login page **(Module-A)** could be validated, only if it is directed to the home page **(Module-B)**, based on the valid and correct inputs.

But, on the non-availability of the **Module-B**, it will not be possible to test **module-A**. These types of circumstances, introduces the stubs & drivers in the **process of software testing**.

A dummy module, **representing the basic functionality or feature of the module-B**, is being developed, and thereafter, it is being integrated with the module-A, to perform testing, efficiently.

This **dummy module is called as stub**, are used to test modules and are created by the team of testers during the process of **top-down integration testing**.

With the assistance of these test stubs, testers are capable of **stimulating the behavior of the lower level modules that are not yet integrated with the software or are missing**.

Use of drivers

Similarly, **drivers**, are used to fulfil the requirements of other modules, such as Log out page (**Module-D**), needs to be directed to the login page **(Module-A)**, after successfully logging out from the application.

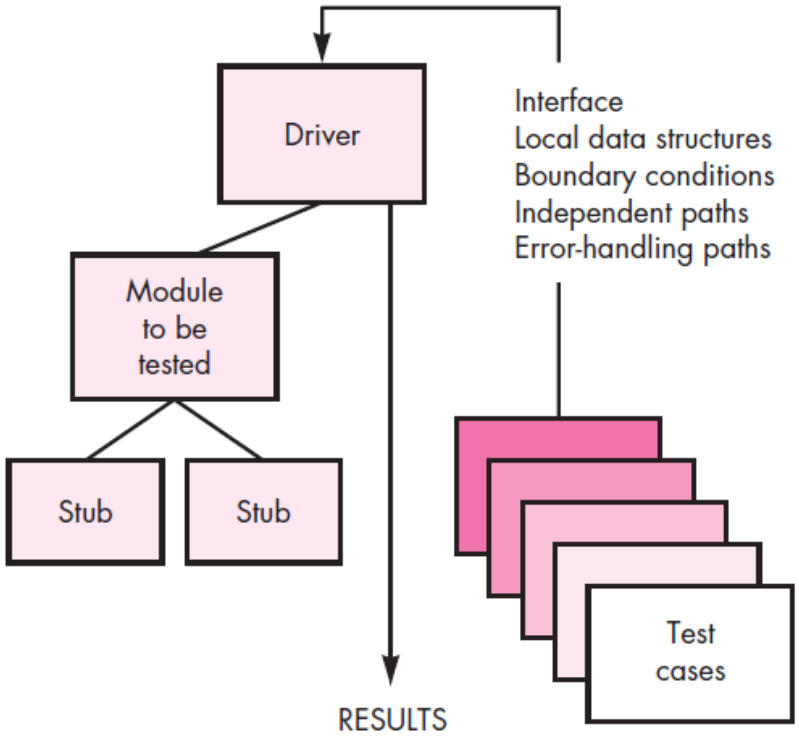
In the event of unavailability of **Module-A**, **drivers** will work as a substitute for it, in order to carry out the testing of module-D.

Drivers, like stubs, are used by software testers to fulfil the requirements of missing or incomplete components and modules. These are usually complex than stubs and are developed during **bottom-up approach of integration testing**.

Drivers can be utilized to test the lower levels of the code, when the upper level of codes or modules are not developed.

Drivers act as pseudo codes that are mainly used when the stub modules are ready, but the primary modules are not ready.

Unit test environment with stub and driver



2. Integration Testing

Integration testing is a systematic technique for constructing the software architecture while at the same time conducting tests to uncover errors associated with interfacing. The objective is to take unit-tested components and build a program structure that has been dictated by design. There is often a tendency to attempt non-incremental integration; that is, to construct the program using a "big bang" approach. All components are combined in advance. The entire program is tested as a whole.

Incremental integration approach is used for integration testing where the program is **constructed and tested in small increments.**

Incremental integration strategies

There are two types of incremental integration approach –

1. top-down integration
2. bottom-up integration

Top-down integration

Top-down integration testing is an incremental approach to construction of the software architecture. Modules are integrated by moving downward through the control hierarchy, beginning with the main control module (main program).

Modules subordinate (and ultimately subordinate) to the main control module are incorporated into the structure in either a **depth-first or breadth-first manner.**

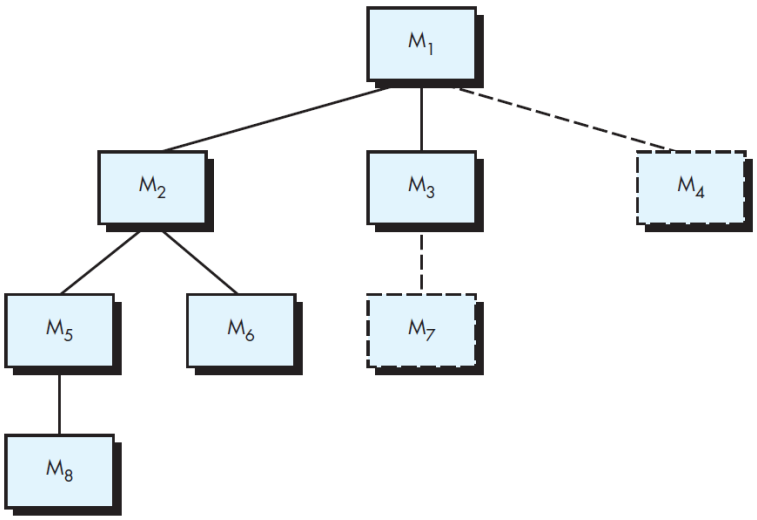
Referring to Figure, **depth-first integration** integrates all components on a major control path of the program structure. Selection of a major path is somewhat arbitrary and depends on application-specific characteristics.

For example, selecting the left-hand path, components M1, M2 , M5 would be integrated first. Next, M8 or (if necessary for proper functioning of M2) M6 would be integrated. Then, the central and right-hand control paths are built.

Breadth-first integration incorporates all components directly subordinate at each level, moving across the structure horizontally. From the figure, components M2, M3, and M4 would be integrated first. The next control level, M5, M6, and so on, follows. The integration process is performed in a series of five steps:

1. The main control module is used as a test driver and stubs are substituted for all components directly subordinate to the main control module.
2. Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.

3. Tests are conducted as each component is integrated.
4. On completion of each set of tests, another stub is replaced with the real component.



The process continues from step 2 until the entire program structure is built. The top-down integration strategy verifies major control or decision points early in the test process.

Bottom-Up Integration

Bottom-up integration testing, as its name implies, begins construction and testing with atomic modules (i.e., components at the lowest levels in the program structure). Because components are integrated from the bottom up, the functionality provided by components subordinate to a given level is always available and the need for stubs is eliminated. A bottom-up integration strategy may be implemented with the following steps:

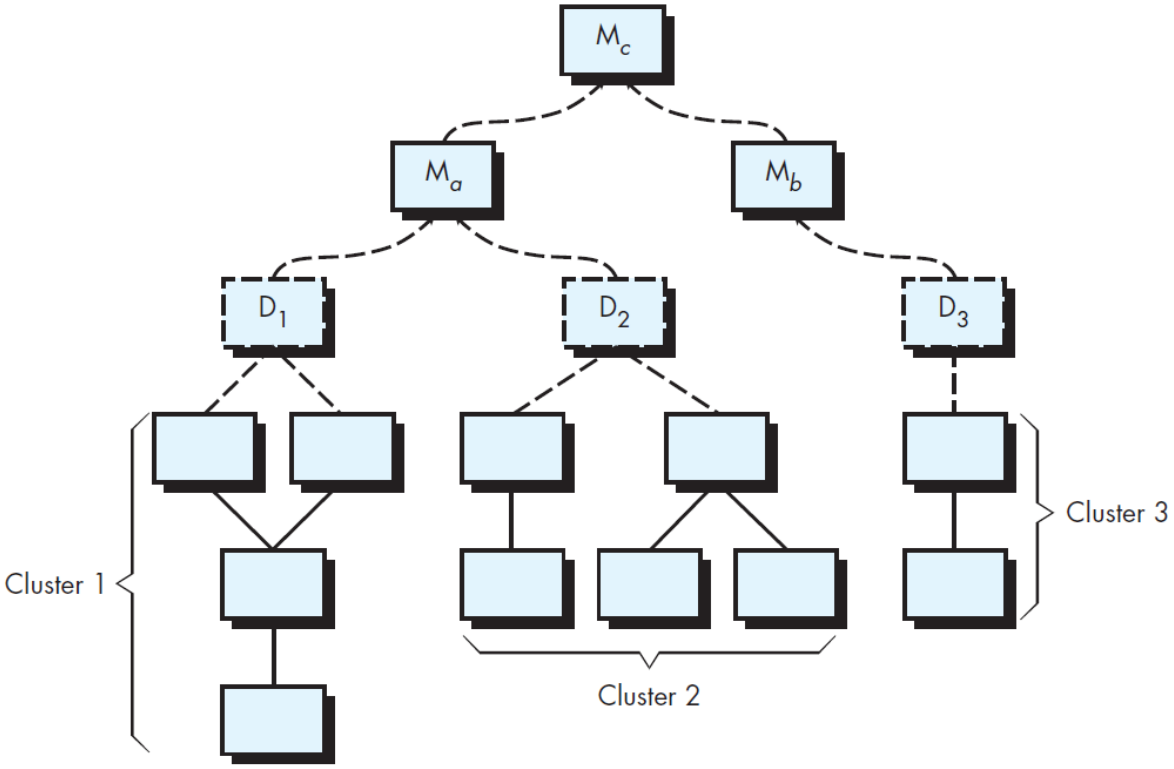
1. Low-level components are combined into clusters (sometimes called builds) that perform a specific software subfunction.

2. A driver (a control program for testing) is written to coordinate test-case input and output.

3. The cluster is tested.

4. Drivers are removed and clusters are combined moving upward in the program structure.

Integration follows the pattern illustrated in Figure. Components are combined to form clusters 1, 2, and 3. Each of the clusters is tested using a driver (shown as a dashed block). Components in clusters 1 and 2 are subordinate to M a. Drivers D1 and D2 are removed and the clusters are interfaced directly to M a. Similarly, driver D3 for cluster 3 is removed prior to integration with module Mb. Both M a and M b will ultimately be integrated with component M c , and so forth.



Regression Testing

Each time a new module is added as part of integration testing the software changes. New data fl ow paths are established, new I/O may occur, and new control logic is invoked. Side effects associated with these changes may cause problems with functions that previously worked flawlessly. In the context of an integration test strategy, regression testing is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects. Regression testing helps to ensure that changes (due to testing or for other reasons) do not introduce unintended behavior or additional errors.

Smoke Testing

Smoke testing is an integration testing approach that is commonly used when product software is developed. It is designed as a pacing mechanism for time-critical projects, allowing the software team to assess the project on a frequent basis. In essence, the smoke-testing approach encompasses the following activities:

1. Software components that have been translated into code are integrated into a build. A build includes all data fi les, libraries, reusable modules, and engineered components that are required to implement one or more product functions.

2. A series of tests is designed to expose errors that will keep the build from properly performing its function. The intent should be to uncover "show-stopper" errors that have the highest likelihood of throwing the software project behind schedule.

3. The build is integrated with other builds, and the entire product (in its current form) is smoke tested daily. The integration approach may be top down or bottom up.

3. Validation Testing

Validation testing begins at the culmination of integration testing, when individual components have been exercised, the **software is completely assembled as a package, and interfacing errors have been uncovered and corrected.**

Validation testing focuses on **user-visible actions** and **user-recognizable output** from the system which are described in **software requirements specification (SRS) document.**

Validation succeeds when software functions in a manner that **can be reasonably expected by the customer.**

Software validation is achieved through a series of tests that demonstrate **conformity with requirements.**

A test plan outlines the classes of tests to be conducted, and a test procedure defines specific test cases that are designed **to ensure that**

- all **functional requirements** are satisfied,
- all **behavioral characteristics** are achieved,
- all **content is accurate and properly presented**,
- all **performance requirements** are attained,
- **documentation is correct**, and
- **usability and other requirements are met** (e.g., transportability, compatibility, error recovery, maintainability).

Acceptance testing as validation by end user

When custom software is built for one customer, a series of **acceptance tests** are conducted to enable the **customer to validate all requirements**.

Acceptance tests are **conducted by the end user** rather than software engineers.

An acceptance test can range from an informal test drive to a planned and systematically executed series of tests.

In fact, acceptance testing can be conducted over a period of weeks or months, thereby **uncovering cumulative errors that might degrade the system over time**.

If software is developed as a product to be used by many customers, it is impractical to perform formal **acceptance tests with each one**.

Types of testing done **by end user as acceptance tests** to uncover errors are-

a. **alpha testing** and
b. **beta testing**

a) Alpha testing

Alpha testing is a **pre-release activity** and one of the types of **acceptance testing**.

Here, testing activity is carried out in a **much-controlled manner** and it is **not accessible by the end-users**.

A newly developed product or updated product undergoes alpha testing **in the testing environment** (at **developer's site**).

Testing is carried out to **simulate real-time behavior** to match the usage of the product by the end-users in the market.

Alpha testing is carried out with either **black-box or white-box** testing techniques or **the combination of both**.

b) Beta Testing

Beta testing is one of the **customer validation** methodologies (**acceptance testing** types), performed by the **end-user** (intended real user) on the product **at the end user's side**.

End-user actually uses it, for over a period of time, **to validate the product for functionality, usability, reliability, and compatibility.**

Product experience gained by the end users are asked for **feedback** on design, functionality, and usability and this helps in assessing the quality of the product.

Difference between Alpha and Beta Testing

Alpha Testing	Beta Testing
Alpha testing is performed **by developers/testers** who are usually internal employees of the organization.	Beta testing is performed **by users/clients** who are not part of the organization.
Alpha testing is performed **at developer's site**.	Beta testing is performed **at end-user's site**.
Alpha testing ensures the quality of the product **before forwarding to beta testing.**	Beta testing also concentrates on the quality of the and ensures that **the product is ready for real time users**.
Alpha testing requires a **testing environment or sample (test) data**.	Beta testing is performed **in live (real world) environment or live data**.
Developers can **immediately address** the critical issues or fixes in alpha testing.	**Feedback collected from beta testing** will be implemented in future versions of the product.
Alpha testing involves both the **white box and black box testing.**	Beta testing commonly uses **black box** testing.

4. System testing

Software is only one element of a larger computer-based system.

Ultimately, software is incorporated with other system elements (e.g., hardware, people, information), and a series of system integration and validation tests are conducted.

System testing **is actually a series of different tests whose primary purpose is to fully exercise the computer-based system.**

Although each test has a different purpose, all work **to verify that system elements have been properly integrated and perform allocated functions**.

Types of system tests for software-based systems

a) Recovery Testing
b) Security Testing
c) Stress Testing
d) Sensitivity Testing
e) Performance Testing
f) Deployment testing

Recovery testing is a system test **that forces the software to fail in a variety of ways and verifies that recovery** is properly performed.

If recovery is **automatic** (performed by the system itself), reinitialization, checkpointing mechanisms, data recovery, and restart are **evaluated for correctness**.

If recovery requires **human intervention**, the **mean-time-to-repair** (MTTR) is evaluated to determine **whether it is within acceptable limits**.

Security testing attempts to verify that protection mechanisms built into a system will, in fact, **protect it from improper penetration**.

Penetration spans a broad range of activities:

- **hackers** who attempt to penetrate systems,
- **disgruntled employees** who attempt to penetrate for revenge,
- **dishonest individuals** who attempt to penetrate for illicit personal gain.

Stress tests are designed to confront programs **with abnormal situations** where essentially, the **tester attempts to break the program.**

Stress testing executes a system in a manner that demands resources in **abnormal quantity, frequency, or volume.**

Sensitivity testing **attempts to uncover data combinations within valid input classes that may cause instability or improper processing.**

Performance testing is designed to **test the run-time performance of software** within the context of an integrated system.

In many cases, software **must execute on a variety of platforms and under more one operating system environment.**

Deployment testing, sometimes called **configuration testing**, **exercises the software in each environment in which it is to operate.**

In addition, deployment testing examines **all installation procedures and specialized installation software** (e.g., installers) that will be used by customers, and all documentation that will be used to introduce the software to end users.

Software testing fundamentals

Characteristics of good test

1. **A good test has a high probability of finding an error.**
   To achieve this goal, the tester must understand the software and attempt to develop a mental picture of how the software might fail.
2. **A good test is not redundant.**
   Testing time and resources are limited. Every test should have a different purpose (even if it is subtly different) and not the same purpose as another test.
3. **A good test should be best of breed.**
   In a group of tests that have a similar intent, time and resource limitations, the test that has the highest likelihood of uncovering a whole class of errors should be used.
4. **A good test should be neither too simple nor too complex.**

White-box testing (Glass-box testing)

White-box testing or glass-box testing, is a test-case design philosophy that **uses the control structure described as part of component-level design to derive test cases**.

Using white-box testing methods, test cases **can be derived that**

(1) guarantee that **all independent paths within a module have been exercised at least once,**

(2) exercise **all logical decisions** on their true and false sides,

(3) execute **all loops** at their boundaries and within their operational bounds, and

(4) exercise **internal data structures** to ensure their validity.

White-Box Testing techniques

1) Basis Path Testing
2) Control Structure Testing

1) Basis Path Testing

Basis path testing is a white-box testing technique first proposed by Tom McCabe.

The basis path testing method enables the test-case designer **to derive a logical complexity measure of a procedural design.**

This measure is used as a guide for defining a **basis set of execution paths**.

Test cases derived to exercise the basis set are **guaranteed to execute every statement in the program at least one-time during testing.**

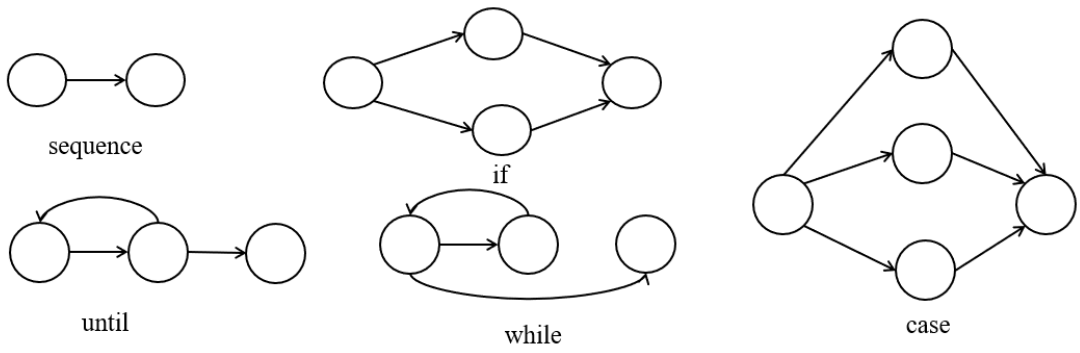
Flow Graph Notation

The basis path method **uses flow graph (or program graph)**.

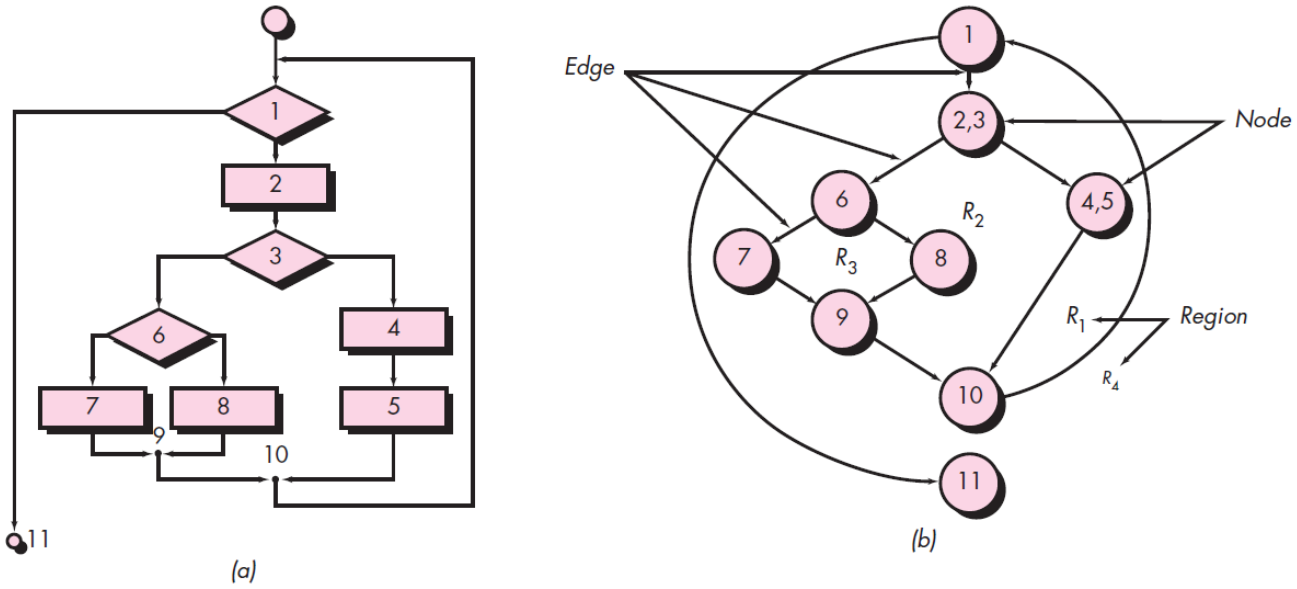
The flow graph represents **logical control flow in program** using the notations.

Different flow graph symbols are used to represent control structures (if, while, until & case) used in program.

The notations used in the flow graph are



Flowchart in figure a can be mapped into a corresponding **flow graph** in figure b as-



Areas bounded by edges and nodes are called regions. When counting regions, the area outside the graph is considered as a region.

Independent Program Paths

An **independent path** is any path through the program that introduces **at least one new set of processing statements or a new condition**.

In flow graph, an independent path must move along **at least one edge** that has not been **traversed before the path** is defined.

Each new path introduces **a new edge**.

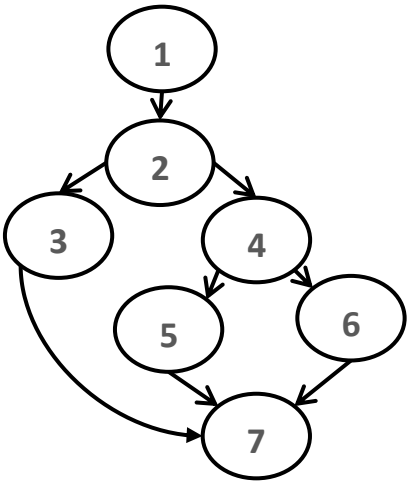
Basis set for the flow graph

A set of independent paths for the flow graph is **basis set for that flow graph**.

That is, while designing tests **to execute these paths** (a basis set),

- every statement in the program will have been guaranteed **to be executed at least one time** and
- every condition will have been executed **on its true and false sides**.

For example, a set of independent paths for the flow graph shown below is



- **Path 1**: through nodes **1-2-3-7**
- **Path 2**: through nodes **1-2-4-5-7**
- **Path 3**: through nodes **1-2-4-6-7**

Thus, paths 1, 2 & 3 constitute a **basis set for the given flow graph**.

Cyclomatic Complexity

The computation of cyclomatic complexity provides the answer **how many paths** to look for.

Cyclomatic complexity is a software metric that provides a quantitative measure of the logical complexity of a program.

When used in the context of the basis path testing method, the value computed for cyclomatic complexity **defines the number of independent paths in the basis set** of a program and provides you with an **upper bound for the number of tests** that must be conducted to ensure that all statements have been executed at least once.

Cyclomatic complexity has a foundation in **graph theory** and provides you with an extremely useful software metric.

Cyclomatic complexity is computed in one of three ways:

1. The number of regions of the flow graph corresponds to the cyclomatic complexity.

   V(G) = Number of regions

2. Cyclomatic complexity V(G) for a flow graph G is

V(G) = E - N + 2

where E is the number of flow graph edges and N is the number of flow graph nodes.

3. Cyclomatic complexity V(G) for a flow graph G is

V(G) = P + 1

where P is the number of predicate nodes contained in the flow graph G.

Referring once more to the same flow graph shown above, the cyclomatic complexity can be computed as:

1. V(G) = **3** regions. (R1,R2,R3)

2. V(G) = 8 edges - 7 nodes + 2 = **3**.

3. V(G) = 2 predicate nodes +1 = **3** (nodes- 2,4)

Therefore, the cyclomatic complexity of the flow graph is 3.

Black box testing

Black-box testing focuses on the **functional requirements of the software.**

That is, black-box testing techniques enable tester **to derive sets of input conditions** that will fully exercise **all functional requirements** for a program.

Black-box testing **attempts to find errors in the following categories**:

- incorrect or missing functions,
- interface errors,
- errors in data structures or external database access,
- behavior or performance errors, and
- initialization and termination errors.

Black-box testing types-

1. Graph based testing
2. Equivalence partitioning
3. Boundary value analysis (BVA)

1. Graph-based testing methods

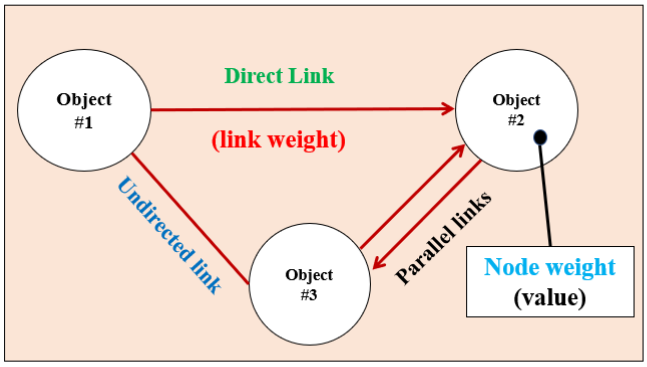
The first step in black-box testing is to **understand the objects** in software and **the relationships** that connect these objects.

Once this has been accomplished, the next step is to **define a series of tests that verify "all objects have the expected relationship to one another".**

Graph-Based testing

- begins by **creating a graph** of important objects and their relationships and
- then **devising a series of tests that will cover the graph** so that **each object and** relationship is exercised and errors are uncovered.

The **symbolic representation of a graph** is shown below,

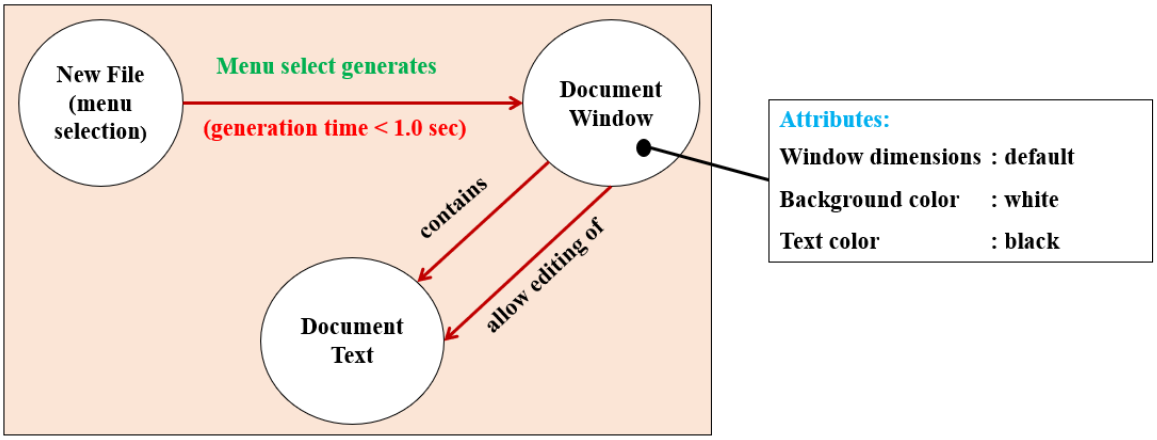


A **graph** contains

- a collection of **nodes** that represent objects,
- **links** that represent the relationships between objects,
- **link weights** that describe some characteristic of a link and
- **node weights** that describe the properties of a node.

An **undirected link** establishes a symmetric relationship between objects.

Consider a portion of a graph for a word-processing application where objects are



2. Equivalence Partitioning

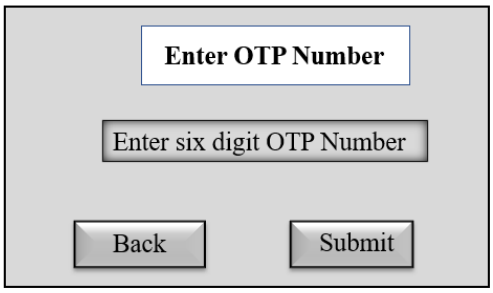
Equivalence partitioning is a black-box testing method **that divides the input domain of a program into classes of data from which test cases can be derived.**

An equivalence class represents a **set of valid or invalid states for input conditions**.

Example of Equivalence Partitioning technique

Assume that there is a function of a software application **that accepts a particular number of digits, not greater and less than that particular number.**

For example, an OTP number which contains only six digits, less or more than six digits will not be accepted, and the application will redirect the user to the error page.



Test Case-1	Test Case-2	Test Case-3	Test Case-4
Digits >=7	Digits <=5	Digits =6	Digits =6
9793765	9765	979376	653209
Invalid	**Invalid**	**Valid**	**Valid**

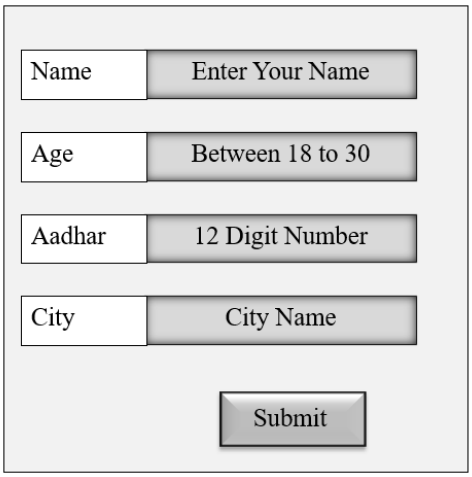
3. Boundary Value Analysis (BVA)

Boundary value analysis leads to a selection of test cases **that exercise bounding values**.

Boundary value analysis is a test-case design technique **that complements equivalence partitioning**. Rather than selecting any element of an equivalence class, BVA leads to the selection of **test cases at the "edges" of the class**.

Tester develops test cases **for both valid and invalid partitions** to capture the behavior of the system on different input conditions.

Consider an application where for age of a person is to be entered. The **minimum value is 18** and **the maximum value is 30, both 18 and 30 will be considered as boundary values**.



The basic assumption of boundary value analysis is, the test cases that are created using boundary values are most likely to cause an error.

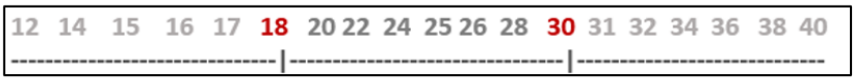
Test cases are developed for each and every value of the range.

Testing of boundary values is done **by making valid and invalid partitions**.

Invalid partitions are tested because testing of output in adverse condition is also essential.

The **valid partition** consists of **boundary values** 18 (minimum), 30 (maximum) and the other values like 19, 20, 21, 22, 23, 24, 25, 26, 27, 28 and 29.

The **invalid partition** consists of the numbers which are less than 18 and more than 30.



Invalid partition	Valid partition	Invalid partition
Invalid test cases	**Valid test cases**	**Invalid test cases**
12,13,17	**19,21,24,27,28,30**	**31,34,37,45**

The software system will be **passed in the test** if it accepts a valid number and gives the desired output, if it is not, then it is **unsuccessful.**

Difference between black box and white box testing

Black Box Testing	White Box Testing
It is a way of software testing in which the internal structure or the program or the code is **hidden** and nothing is known about it.	It is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
It is mostly done **by software testers**.	It is mostly done **by software developers**.
It can be referred as **external** software testing.	It is the **internal** software testing.
It is **functional or behavior testing** of the software.	It is **structural or logic testing** of the software.
No knowledge of programming is required.	It is mandatory to have knowledge of programming.

Software Maintenance

Software maintenance is the **periodic modification of software to suit the changing needs of users, organizations or environment.**

The **definition of software maintenance** by IEEE is –

"The **software maintenance** of a software product after the delivery to correct faults **to improve performance or other attributes, or to adapt the product to a modified environment**".

It is observed that maintenance consumes about **50% to 80% of total life cycle costs**.

Types of software maintenance

1. Corrective Maintenance:

It deals with **repair of faults or defects that are not detected during testing but found later when the software is in actual use**.

A software application may perform correctly in most cases but may fail in some rare exceptional conditions.

The need for corrective maintenance is usually initiated by **bug reports drawn up by the end users.**

Corrective maintenance deals with fixing bugs in the code.

2. Adaptive Maintenance:

Adaptive software maintenance becomes important when the environment of your software changes. It consists of **adapting software to changes in the environment.**

The term environment refers to the **totality of all conditions that act from outside upon the system.**

This can be brought on by **changes to the operating system, hardware, software dependencies, Cloud storage, or even changes within the operating system**.

Sometimes, adaptive software maintenance reflects **organizational policies or rules** as well. **Updating services, making modifications to vendors, or changing payment processors can all necessitate adaptive software maintenance**.

3. Perfective Maintenance:

Perfective software maintenance **focuses on the evolution of requirements and features that existing in software system.**

As users interact with applications, they may notice things that are **missing or suggest new features** that they would like as part of the software, which could become future projects or enhancements.

Perfective software maintenance takes over some of the work, **both adding features that can enhance user experience and removing features that are not effective and functional**.

Examples-

- Face recognition/biometric attendance in place of manual entry,

4. Preventive Maintenance:

Preventative software maintenance **helps to make changes and adaptations to software so that it can work for a longer period of time.**

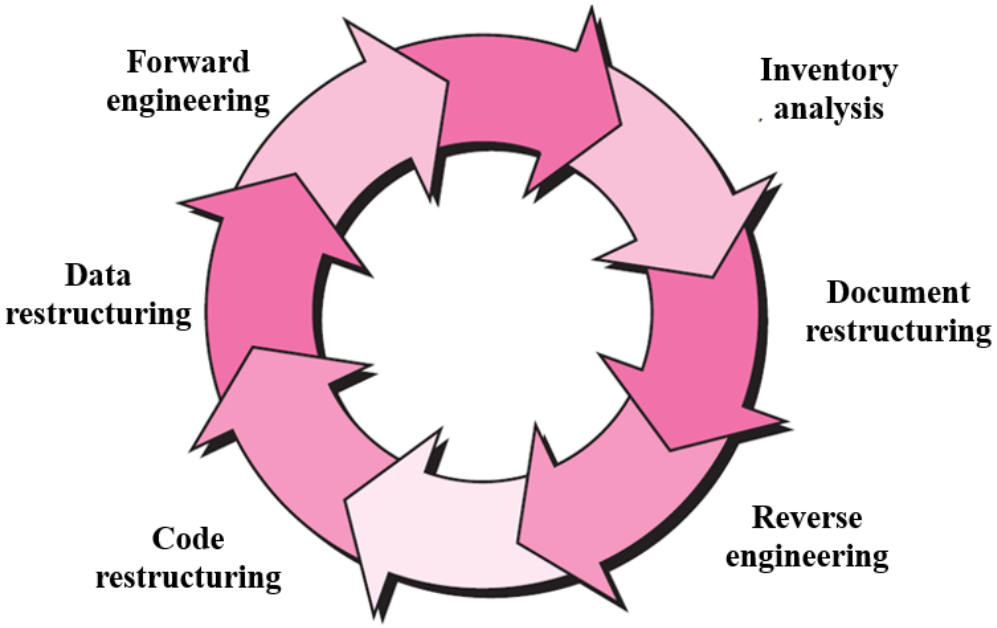
The focus of this type of maintenance is **to prevent the deterioration of software as it continues to adapt and change.** These services can include **optimizing code and updating documentation as needed.**

Preventative software maintenance **helps to reduce the risk associated with operating software for a long time, helping it to become more stable, understandable, and maintainable**.

Software Re-engineering

Software reengineering can be applied for software maintenance problems.

Reengineering is a **rebuilding activity**.



Reverse engineering

Reverse engineering for software is the process of analyzing a program in an **effort to create a representation of the program** at a higher level of abstraction than source code.

Reverse engineering is a **process of design recovery**.

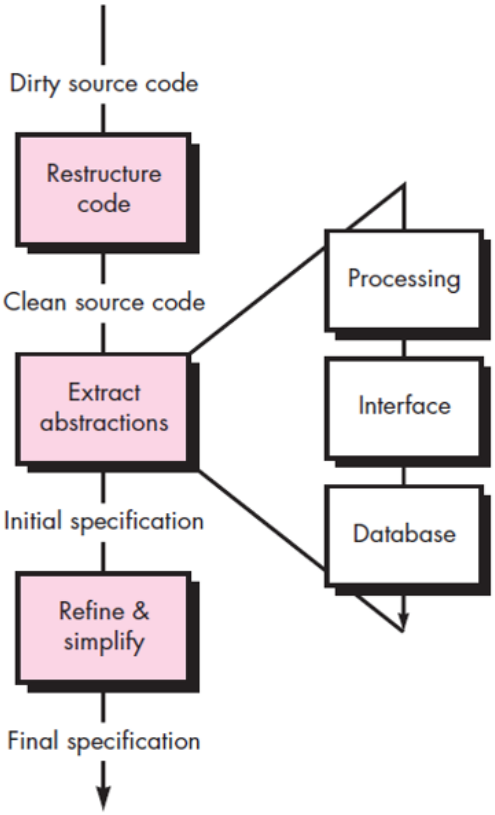
Reverse engineering tools **extract data, architectural, and procedural design information from an existing program.**

Reverse Engineering Process

Unstructured ("dirty") source code is restructured so that it contains only the structured programming constructs. This makes the source code easier to read and provides the basis for all the subsequent reverse engineering activities.

In **extract abstractions,**

- old program is processed to develop a meaningful specification of the processing that is performed,
- the user interface that is applied, and
- the program data structures or database that is used.



References:

- Roger Pressman, Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill 2009)
- **https://www.geeksforgeeks.org/**
- **https://www.javatpoint.com/**
- **https://www.softwaretestinghelp.com/**
- **https://www.guru99.com/**